

Technical Task: Agentic Scene Evaluator & Self-Correction Loop

Timebox: 2-3 hours max. Since the focus is heavily on your architecture and system design rather than a perfectly polished app, just submit whatever you have done by the mark, even if it isn't totally complete.

Objective: Build a closed-loop Python pipeline (script or FastAPI) that autonomously generates text-to-image prompts and refines them based on Vision Language Model (VLM) critiques.

- **What you will build:** The routing code that connects an LLM and a VLM. Your code must generate a text-to-image prompt, pass it along with a dummy image to a VLM for a critique, and build the logic loop that feeds that critique back to the LLM for a revision.
- **What we provide:** The API key, the raw scene text, the mock data specifications, and the dummy test image to be graded. (*Note: You do not need to generate actual images, train models, or build a database*).

System Requirements

1. Context Retrieval (Tool Use)

- Write a mock function `get_character_metadata(character_name)` that returns hardcoded JSON.
- Provide a raw scene text to an LLM. The LLM must autonomously use the mock tool to fetch character details before proceeding.

2. The Initial Generation

- Using the raw scene and retrieved metadata, the LLM must generate an initial text-to-image prompt.

3. The VLM Critic

- Pass the generated prompt and a provided test image to a VLM (acting as a "Continuity Supervisor").
- The VLM must evaluate the image against the text constraints and output a critique strictly in this JSON schema:

```
{  
  "overall_match_score": 0,  
  "missing_elements": ["string"],  
  "hallucinated_elements": ["string"],  
}
```

```
"actionable_feedback": "string"
}
```

4. The Self-Correction Loop (Core Challenge)

- If the `overall_match_score` is below 90, your system must automatically route the `actionable_feedback` back to the original LLM.
- The LLM must ingest the critique and output a *revised* text-to-image prompt.
- *Constraint:* Cap the loop at a maximum of 2 iterations to prevent infinite loops.

5. Error Handling & State Management

- The system must handle LLM/VLM timeouts and JSON schema failures gracefully (e.g., retry logic).
- Log the state of the loop to the console (Initial Prompt -> VLM Score -> Revised Prompt).

Starter Data & API Key

- **OpenAI API Key:**
`[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]`
- **Raw Scene Text:** *"EXT. ALLEYWAY - NIGHT. A torrential downpour. Kael stands under a flickering neon sign. He is wearing a formal suit, completely soaked, and holds a vintage brass compass."*
- **Mock Tool Data:** When queried for "Kael", return: `{"age": 30, "wardrobe": "formal suit", "prop": "brass compass"}`
- **Test Image URL:**
`https://images.unsplash.com/photo-1509822929063-6b6cfc9b42f2` (Use this, or any other dummy image for the VLM to grade).

Deliverables

1. **Code:** GitHub repo link or a zipped folder containing your code.
2. **Docs:** A brief README with instructions on how to run it locally.
3. **Architectural Reflection:** In your README, write a brief paragraph answering: *How would you architect this exact autonomous loop to process 10,000 scenes asynchronously in a cloud environment without bottlenecking on VLM API rate limits?*